Implementasi WordCount dengan PySpark

Pembuktian pemrosesan paralel: data masuk → partition → transformation → action → pengaruh partition terhadap performa

Mata kuliah: Analisis Pemrosesan Data Paralel | **Nama:** Andi Putra Malba Syaggaf | **NIM:** 312410370

1. Pendahuluan dan Tujuan

Laporan ini bukan sekadar menjalankan program WordCount, tetapi **membuktikan** bagaimana Apache Spark memproses data secara paralel: (1) bagaimana data dibagi menjadi *partition*, (2) bagaimana *transformation* disusun menjadi DAG yang dibagi menjadi *stage* pada batas *shuffle*, (3) bagaimana *action* memicu *job* yang dijalankan sebagai *task* (satu task per partition), dan (4) bagaimana jumlah partition memengaruhi waktu eksekusi. Semua angka pada laporan ini adalah **hasil pengukuran nyata** dari kode yang disertakan (`wordcount_pyspark.py`, `benchmark.py`), bukan angka contoh.

2. Lingkungan dan Dataset

Komponen	Nilai	Komponen	Nilai
Apache Spark	4.2.0 (mode *local[*]*)	Dataset	bigdata.txt, 100.000 baris (8,2 MB)
Python / Java	3.12 / OpenJDK 21	Total token	1.298.900 kata (setelah preprocessing)
CPU / RAM	**1 core** / 4 GB	Kata unik	148 kata

Dataset. Teks berbahasa Indonesia sintetis yang dibuat oleh `generate_dataset.py` (seed 42) dengan distribusi frekuensi kata ala Zipf (kata fungsi seperti *yang, dan, di* sangat sering, kata isi jarang), 8–18 kata per baris, diberi huruf kapital dan tanda baca agar tahap preprocessing bermakna. Ukurannya 20× lipat dari syarat minimal 5.000 baris supaya waktu eksekusi cukup besar untuk diukur. **Catatan keterbukaan:** dataset sintetis, bukan korpus nyata; hanya 148 kata unik, sehingga beberapa temuan (partition kosong, skew) dipengaruhi karakter data ini.

Hasil: 10 kata terbanyak (keluaran program; diverifikasi identik dengan hitungan Python murni untuk P = 1, 8, dan 32):

#	Kata	Frekuensi	%
1	yang	183.399	14,12
2	dan	97.727	7,52
3	di	67.919	5,23
4	untuk	52.675	4,06
5	dengan	43.042	3,31

#	Kata	Frekuensi	%
6	dalam	37.126	2,86
7	pada	31.633	2,44
8	ini	28.150	2,17
9	itu	25.395	1,96
10	dari	22.802	1,76

Kata *yang* saja menyumbang 14,1% dari seluruh token. Fakta ini penting: satu kunci yang dominan akan menjadi sumber *data skew* (Bagian 6).

3. Implementasi dan Penjelasan Transformation / Action

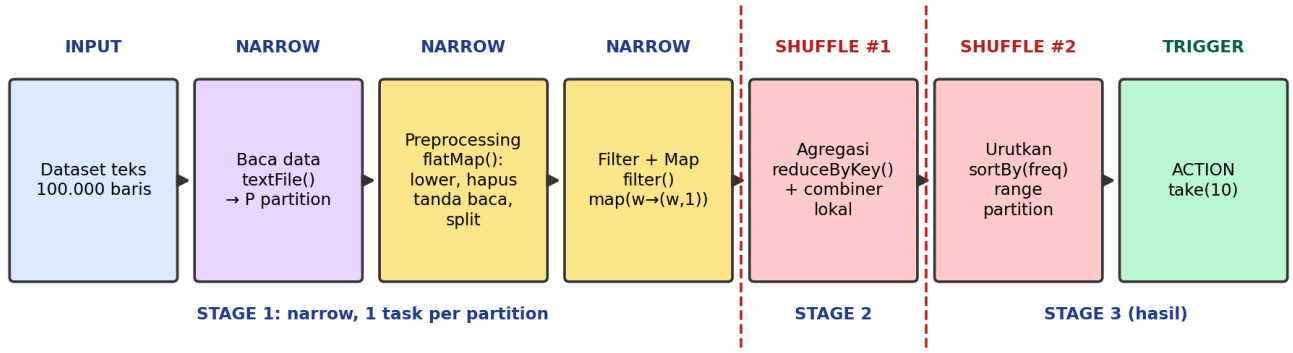
Program memakai RDD API agar setiap langkah paralelnya terlihat jelas. Inti kodenya (lengkap di `wordcount_pyspark.py`):

```
text  = sc.textFile(path, minPartitions=P).repartition(P)             # data masuk -> P partition
words = text.flatMap(lambda l: re.sub(r"[^\w\s]", " ", l.lower()).split())  # preprocessing
words = words.filter(lambda w: w and not w.isdigit())                   # buang token kosong/angka
pairs = words.map(lambda w: (w, 1))                                     # (kata, 1)
counts = pairs.reduceByKey(lambda a, b: a + b, numPartitions=P)         # agregasi (shuffle #2)
top10  = counts.sortBy(lambda kv: kv[1], ascending=False).take(10)      # urut + ACTION
```

Operasi	Jenis	Peran dalam pipeline dan pengaruhnya pada paralelisme
textFile(path, minPartitions=P)	Transformation (lazy)	Membentuk RDD dari berkas; Spark memecah berkas menjadi split sehingga tiap split menjadi satu partition. Belum ada data yang dibaca.
repartition(P)	Wide (shuffle #1)	Mengacak ulang data merata ke tepat P partition. Dipakai hanya agar eksperimen terkontrol, bukan bagian logika WordCount.
flatMap(preprocess)	Narrow	Satu baris menjadi banyak kata (lowercase, hapus tanda baca, split). Berjalan lokal di tiap partition, tanpa pindah data.

Operasi	Jenis	Peran dalam pipeline dan pengaruhnya pada paralelisme
filter(...) dan map(w→(w,1))	Narrow	Membuang token kosong/angka dan membentuk pasangan kunci-nilai. Digabung (pipelining) dengan flatMap dalam satu stage.
reduceByKey(+, P)	Wide (shuffle #2)	Menjumlahkan per kata. Ada *combiner* di sisi map: tiap task menjumlahkan lokal lebih dulu sehingga hanya ≤148 pasangan per task yang dikirim.
sortBy(freq)	Wide (shuffle #3)	Mengurutkan global lewat *range partitioning*. Spark mengambil sampel data lebih dulu, sehingga memicu job tambahan walau tampak sebagai transformation.
take(10)	**ACTION**	Memicu eksekusi seluruh DAG sebagai satu job; mengembalikan 10 elemen ke driver (lebih aman daripada collect()).
mapPartitionsWithIndex+ collect, count()	Narrow / ACTION	Diagnostik: menghitung baris per partition dan total token/kata unik.

Gambar 1. Pipeline pemrosesan data WordCount



Garis putus-putus merah = batas stage (shuffle: data dipindah antar-executor lewat disk/jaringan)

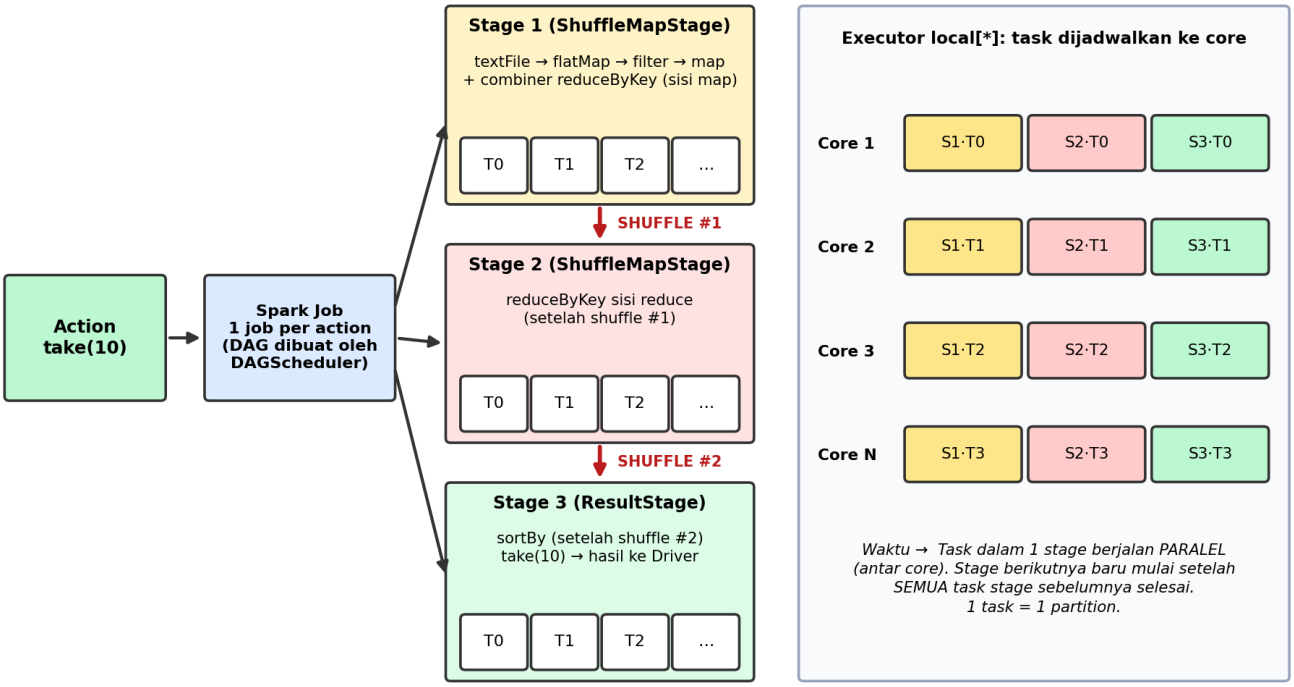
Pipeline dibaca dari kiri ke kanan. Operasi narrow digabung dalam satu stage; setiap wide (shuffle) memotong stage baru.

4. Bagaimana Data Diproses Secara Paralel

a) Data masuk dan pembagian partition. Dengan P = 8, hasil diagnostik menunjukkan 100.000 baris terbagi hampir rata: `{0: 12516, 1: 12490, 2: 12510, 3: 12506, 4: 12499, 5: 12489, 6: 12490, 7: 12500}` (selisih terbesar-terkecil hanya 27 baris atau 0,2%). Karena satu task memproses satu partition, delapan partition berarti delapan task yang *dapat* berjalan bersamaan bila tersedia delapan core. Pembagian merata ini penting: stage baru selesai saat task **paling lambat** selesai.

b) Action → Job → Stage → Task. Transformation hanya mencatat *lineage* (DAG); tidak ada komputasi sampai ada action. Pada `toDebugString()` terlihat tiga batas shuffle (repartition, reduceByKey, sortBy). Pohon eksekusi pada Gambar 2 disederhanakan tanpa *repartition*; pada program sebenarnya ada satu stage tambahan di depan.

Gambar 2. Alur Spark Job → Stage → Task



Task dalam satu stage berjalan paralel; antar-stage ada barrier karena stage berikutnya membutuhkan seluruh keluaran shuffle stage sebelumnya.

Kelompok (Job)	Stage → jumlah task (hasil statusTracker)	Penjelasan
A. Diagnostik (Job 0)	Stage 0, 1: masing-masing 8 task, dijalankan	Action collect() pertama: membaca data + repartition; memaksa pembagian partition terjadi.
B. Sampling sortBy (Job 1, 2)	Stage 2–7: 8 task per stage; sebagian dijalankan, sebagian *skipped*	RangePartitioner menarik sampel kunci untuk menentukan batas rentang. Ini biaya tersembunyi dari sortBy.
C. Action take(10) (Job 3)	Stage 8, 9: 8 task, **skipped**; Stage 10: 8 task; Stage 11: **1 task**	Shuffle yang sudah ada dipakai ulang (skipped). Stage hasil hanya 1 task: take() memindai partition satu per satu dan 10 kata teratas sudah ada di partition pertama hasil sort menurun.

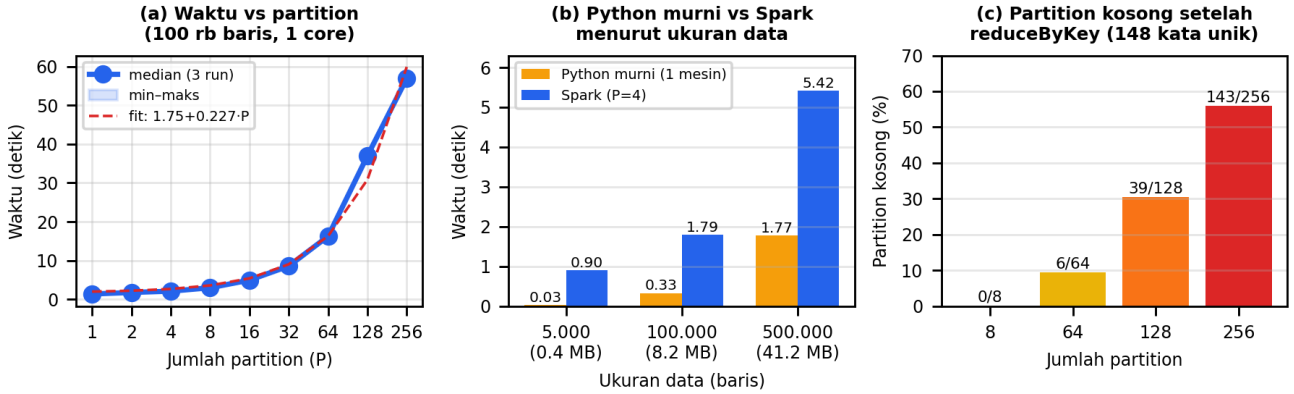
Bukti ini menjawab pertanyaan inti: jumlah task per stage sama dengan jumlah partition (8), job dibuat per action, dan Spark menghindari kerja ulang dengan memakai kembali berkas shuffle (*stage skipped*).

5. Analisis Pengaruh Jumlah Partition terhadap Performa

Metode: pipeline lengkap (baca → preprocessing → reduceByKey → sortBy → take) dijalankan pada dataset 100.000 baris untuk P = 1…256, setelah *warm-up* JVM, dengan 3 kali pengulangan per konfigurasi (median dan rentang dilaporkan). Waktu di sini mencakup job sampling sortBy, sehingga lebih besar dari waktu action tunggal pada Bagian 4.

Partition (P)	1	2	4	8	16	32	64	128	256
Median (s)	1,36	1,75	2,09	2,97	4,93	8,58	16,23	36,97	56,95
Min–maks (s)	1,2–1,4	1,7–2,0	2,0–2,3	2,9–3,0	4,8–5,0	8,1–8,9	16,2–16,6	36,6–37,3	56,5–58,5
Relatif thd P=1	1,0×	1,3×	1,5×	2,2×	3,6×	6,3×	12,0×	27,2×	42,0×

Gambar 3. Hasil eksperimen terukur



Gambar 3a: waktu naik monoton seiring bertambahnya partition; P = 256 kira-kira 42 kali lebih lambat daripada P = 1. Gambar 3b dibahas pada Bagian 6.

Interpretasi. Hasil ini **berbeda dari tabel contoh di lembar tugas** (yang menurun sampai P = 8 lalu naik) dan perbedaannya bisa dijelaskan: mesin uji hanya memiliki **1 core**. Menambah partition hanya berguna bila ada core yang benar-benar menganggur untuk menjalankan task secara bersamaan. Dengan 1 core, semua task tetap berjalan bergiliran (indikator progres Spark selama eksperimen menampilkan pola '7 selesai + 1 berjalan' dari 8 task, yaitu hanya satu task aktif pada satu waktu), sehingga yang tersisa hanya biaya tambahannya. Regresi linear terhadap data menghasilkan $T(P) \approx 1.75 + 0.227 \cdot P$ detik (R^2 = 0,98): setiap partition tambahan menambah sekitar 0.23 detik pada pipeline ini. Penyebab yang masuk akal dan konsisten dengan data:

- **Overhead per task:** penjadwalan, serialisasi *closure*, pembuatan berkas shuffle, dan komunikasi JVM–Python dibayar untuk setiap task di setiap stage.
- **Jumlah blok shuffle M×R:** setiap reducer mengambil blok dari setiap mapper, sehingga ada M×R blok (P = 256 berarti 65.536 blok per shuffle). Namun kenaikan waktu per partition pada data ini relatif stabil (sekitar 0,16–0,32 detik), bukan kuadrat, sehingga kontribusi faktor ini tidak dapat saya pastikan.
- **Partition kosong:** hanya ada 148 kata unik, sehingga pada P = 256 sebanyak 143 partition (56%) kosong setelah reduceByKey: task dijalankan tanpa pekerjaan berguna (Gambar 3c).

Saya tidak mengisolasi kontribusi masing-masing faktor secara terpisah; ketiganya adalah penjelasan yang mungkin, bukan hasil pemisahan eksperimen. **Implikasi pada cluster nyata:** secara konseptual $T(P) \approx W / \min(P, C) + c \cdot P$ dengan C = jumlah core total, sehingga kurva berbentuk lembah dengan titik optimum di sekitar P ≈ C sampai beberapa kali C (panduan *Tuning* Spark menyarankan 2–3 task per core). Bagian turun pada kurva tersebut **tidak dapat saya buktikan pada mesin 1-core ini**; yang terbukti adalah bagian naiknya (biaya kelebihan partition).

6. Single Machine vs Distributed dan Identifikasi Bottleneck

Pengukuran (Gambar 3b). Pembanding single machine adalah program Python murni (`collections.Counter`) pada data dan preprocessing yang sama. Pada 5.000 baris Python selesai dalam 0,03 detik dan Spark 0,90 detik (≈32× lebih lambat); pada 100.000 baris 0,33 s vs 1,79 s (≈5,4×); pada 500.000 baris 1,77 s vs 5,42 s (≈3,1×). Jadi pada data kecil, Spark **kalah** karena biaya tetap (JVM, penjadwalan job, serialisasi) tidak sebanding dengan pekerjaannya; selisih rasio menyempit seiring data membesar karena biaya tetap itu teramortisasi. Titik impas tidak tercapai pada rentang yang diuji (≤ 41 MB, 1 core); saya tidak mengekstrapolasi di luar rentang itu.

Aspek	Single machine (diukur / konsep)	Distributed Spark (diukur / konsep)
Data yang diproses	Dibatasi RAM dan disk satu mesin.	Dibagi ke banyak node; ukuran data bisa melebihi RAM satu mesin (konsep).
Waktu eksekusi	**Lebih cepat pada data kecil** (diukur: 0,03 s vs 0,90 s pada 5.000 baris).	Lebih cepat bila data besar *dan* core banyak (konsep); pada mesin ini belum terbukti.
Skalabilitas	Vertikal (menambah spek mesin), ada batas fisik.	Horizontal (menambah node).
Ketahanan gagal	Mesin mati = pekerjaan hilang.	Task gagal dijadwal ulang; lineage RDD memungkinkan komputasi ulang partition hilang.

Aspek	Single machine (diukur / konsep)	Distributed Spark (diukur / konsep)
Biaya dan kompleksitas	Murah dan sederhana.	Overhead koordinasi, shuffle, konfigurasi cluster (terukur: ≈0,23 s per partition).

Bottleneck yang teridentifikasi pada pipeline ini

- **Shuffle (reduceByKey dan sortBy).** Memindahkan data antar-executor lewat disk/jaringan dan menjadi batas stage. Combiner menekan volumenya: shuffle write reduceByKey hanya 0,021 MB karena tiap task mengirim ≤148 pasangan, bukan sekitar 162 ribu token per task.
- **Data skew.** Jika pasangan dikirim tanpa agregasi lokal. reducer yang menerima kata *yang* menanggung beban jauh lebih besar. Terukur: pada P = 8 reducer terbesar menerima 277.266 record vs rata-rata 162.362 (1.71×); pada P = 32 reducer terbesar 211.694 vs 40.591 (5.22×). Menambah partition justru memperlebar rasio skew karena satu kunci tidak dapat dipecah; satu task menjadi penentu waktu stage.
- **Overhead penjadwalan dan partition kosong** pada P berlebihan (Bagian 5): bottleneck dominan pada eksperimen ini.
- **Biaya tersembunyi sortBy** (job sampling tambahan, Bagian 4) dan **serialisasi Python–JVM** pada PySpark: setiap lambda dipanggil per record di proses Python.
- **reduceByKey vs groupByKey: hasilnya bergantung pada kardinalitas kunci.** Pada 148 kunci, reduceByKey mengirim shuffle lebih sedikit (0,010 vs 0,019 MB) tetapi **lebih lambat** (1,68 s vs 1,15 s); dugaan saya, karena lambda Python dipanggil per record sedangkan groupByKey memakai operasi bawaan. Pada 200.000 kunci (1 juta record), reduceByKey **lebih cepat** (2,42 s vs 3,14 s, ≈23%), dengan volume shuffle hampir sama (4,48 vs 4,64 MB). Jadi keunggulan reduceByKey yang lazim diajarkan tidak otomatis berlaku pada skala kecil di PySpark; penyebab pastinya belum saya verifikasi.

7. Kesimpulan

- **Alur paralel terbukti:** data terbagi menjadi 8 partition yang seimbang (selisih 0,2%); transformation bersifat lazy; action take(10) memicu job; shuffle membagi job menjadi stage; jumlah task per stage sama dengan jumlah partition; stage yang keluarannya sudah ada dilewati (skipped).
- **Jumlah partition bukan 'makin banyak makin cepat'.** Pada mesin 1-core, P = 1 tercepat (1,36 s) dan P = 256 terlambat (56,95 s, 42×). Partition hanya menguntungkan jika ada core untuk menjalankannya bersamaan; di luar itu yang dibayar hanyalah overhead (≈0,23 s per partition; kemungkinan berasal dari overhead task, blok shuffle M×R, dan partition kosong).
- **Spark tidak selalu lebih baik daripada single machine.** Pada data ≤ 41 MB di satu mesin, Python murni 3–32× lebih cepat. Spark layak dipilih ketika data atau komputasi melampaui satu mesin, atau saat butuh toleransi kegagalan.
- **Bottleneck utama** pipeline ini: overhead penjadwalan akibat partition berlebih, shuffle, dan data skew pada kata dominan; secara konseptual dapat dikurangi dengan memilih P ≈ 2–3× jumlah core (panduan Tuning Spark), memakai agregasi lokal, dan menghindari partition kosong; efek perbaikan ini belum diuji pada cluster multi-core.

Keterbatasan. Satu mesin 1-core dengan mode local (bukan cluster sungguhan); dataset sintetis dengan 148 kata unik; 3 pengulangan per konfigurasi. Nilai absolut tidak dapat digeneralisasi; arah tren (biaya overhead, skew, partition kosong) adalah temuan yang kuat. Pengujian di cluster multi-core adalah langkah lanjutan yang paling bernilai.

Referensi

[1] Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient Distributed Datasets: A fault-tolerant abstraction for in-memory cluster computing. *Proc. 9th USENIX NSDI*, 15–28.

[2] Dean, J., & Ghemawat, S. (2004). MapReduce: Simplified data processing on large clusters. *Proc. 6th USENIX OSDI*, 137–150.

[3] Damji, J. S., Wenig, B., Das, T., & Lee, D. (2020). *Learning Spark: Lightning-Fast Data Analytics* (2nd ed.). O'Reilly Media.

[4] Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly Media.

[5] Apache Software Foundation. *RDD Programming Guide* dan *Tuning Spark*. https://spark.apache.org/docs/latest/ (diakses Oktober 2026).